

N.	Requisitos Funcionais	Situação	Implementação
1	Apresentar graficamente menu de opções aos usuários do Jogo, no qual pode se escolher fases, escolher ver colocação (<i>ranking</i>) de jogadores e escolher demais opções pertinentes (previstas nos demais requisitos).	Parcialmente implementado	Pode ser escolhida a fase e sair do jogo, mas falta a opção de 1 ou 2 jogadores e o Ranking.
2	Permitir um ou dois jogadores com representação gráfica aos usuários do Jogo, sendo que no último caso é para que os dois joguem de maneira concomitante.	Sim	
3	Disponibilizar ao menos duas fases <u>distintas</u> , com chão, que podem ser jogadas sequencialmente ou selecionadas, via menu, nas quais jogadores tentam neutralizar inimigos por meio de algum artifício e vice-versa.	Sim	As fases ainda não podem ser jogadas sequencialmente. Falta terminar de implementar as neutralizações.
4	Ter pelo menos três tipos distintos de inimigos, cada qual com sua representação gráfica, sendo que ao menos um deles deve poder lançar projétil contra o(s) jogador(es) e um dos inimigos deve ser um 'chefão'.	Sim	
5	Ter a cada fase ao menos dois tipos de inimigos (<u>um deles exclusivo nela</u>) com número aleatório de instâncias, podendo ser várias instâncias (<u>definindo um máximo – e.g. dez</u>) e sendo pelo menos 3 instâncias <u>para cada tipo que estiver na fase</u> .	Parcialmente	Apenas as posições estão aleatórias.
6	Ter três tipos de obstáculos, cada qual com sua representação gráfica, sendo que ao menos um causa dano em jogador se eles colidirem.	Sim	
7	Ter em cada fase ao menos dois tipos de obstáculos (<u>um deles exclusivo nela</u>) com número aleatório (<u>definindo um máximo</u>) de instâncias (<i>i.e.</i> , objetos), sendo pelo menos 3 instâncias por tipo.	Sim	
8	Ter em cada fase um cenário de jogo constituído por obstáculos, sendo que parte deles devem ser plataformas ou similares, sobre as quais pode haver inimigos e podem subir jogadores. Em cada fase, só pode ter um tipo coincidente de inimigo e um tipo coincidente de obstáculo (que é a plataforma) em relação às demais fases.	Sim	
9	Gerenciar colisões entre jogador para com os inimigos e seu conjunto de projéteis, bem como entre jogador para com os obstáculos. Ainda, todos eles devem sofrer o efeito de alguma 'gravidade' no âmbito deste jogo de plataforma vertical e 2D.	Sim	

10	Permitir: (1) salvar nome do usuário, manter/salvar pontuação (incrementada via neutralização de inimigos) do jogador controlado pelo usuário e gerar lista de pontuação (<i>ranking</i>). E (2) Pausar e Salvar/Recuperar Jogada.	NÃO realizado.	
Total de requisitos funcionais apropriadamente realizados. (Cada tópico realizado efetivamente vale 10%)			
Os requisitos dependem em algo uns dos outros, na chamada interdependência de requisitos.			

N.	Conceitos	Uso	O quê / Onde & Justificativa sucinta em uma frase
1	Elementares:		
1.1 &	- Classes, objetos. & - Atributos (privados), variáveis e constantes. - Métodos (com e sem retorno).	Sim	- Todos .h e .cpp. - Classes, Objetos, Atributos e Métodos foram utilizados porque são conceitos elementares na orientação a objetos.
1.2 &	- Métodos (com retorno <i>const</i> e parâmetro <i>const</i>). - Construtores (sem/com parâmetros) e destrutores	Sim	- Na maioria dos .h e .cpp. - A constância pertinente evita mudanças equivocadas, construtores são mandatórios para inicializar atributos e destrutores pertinentes para finalizações como desalocações.
1.3	- Classe Principal.	Sim	- Jogo.cpp/Jogo.hpp. - Uma classe Principal é mais 'purista' em termos de OO.
1.4	- Divisão em .h e .cpp.	Sim	- No desenvolvimento como um todo. - Permite organizar as classes e afins que compõem o sistema.
2	Relações de:		
2.1	- Associação direcional. & - Associação bidirecional.	Sim	- Em vários dos .h e .cpp, como entre a classe Jogo e Menu
2.2 &	- Agregação via associação. - Agregação propriamente dita.	Sim	- Em vários dos .h e .cpp, como nas classes nos <i>namespaces</i> W e Z. - ...
2.3 &	- Herança elementar. - Herança em vários níveis.	Sim	- Em alguns dos .h e .cpp. Exemplo: Fase_prim como herança elementar, inimigos como herança em vários níveis - ...
2.4	- Herança múltipla pertinente.	Não	
3	Ponteiros, generalizações e exceções		
3.1	- Operador <i>this</i> para fins de relacionamento bidirecional	Sim	- Precisamente nos .h e .cpp da classe Jogo e Menu, ao realizar associação bidirecional.
3.2	- Alocação de memória (<i>new</i> & <i>delete</i>).	Sim	Presente na maioria dos .cpp e .hpp, por exemplo na classe Fase_prim
3.3	- Gabaritos/ <i>Templates</i> criada/adaptados pelos autores para Listas.	Sim	Na classe lista, que é utilizada pela classe ListaElementos
3.4	- Uso de Tratamento de Exceções (<i>try catch</i>).	Sim	Na classe Fase_prim, para abrir o arquivo .txt que contém as informações dos inimigos
4	Sobrecarga de:		
4.1	- Construtoras e Métodos.	parcialmente	Aplicada na construtora da classe Ente.
4.2	- Operadores (2 tipos de operadores pelo menos).	Sim	
---	Persistência de Objetos (via arquivo de texto ou binário)		
4.3	- Persistência de Objetos.	Sim	...
4.4	- Persistência de Relacionamento de Objetos.	Não	...

5	Virtualidade:		
5.1	- Métodos Virtuais Usuais.	Sim	Classe Personagens, que possui o método mover como virtual, que aplica gravidade a todos os Personagens
5.2	- Polimorfismo extensivamente à luz do padrão arquitetural (i.e., diagrama de classes assaz genérico) dado.	Sim	Classes Entidades, Personagens, Inimigos...
5.3	- Métodos Virtuais Puros / Classes Abstratas à luz do padrão arquitetural.	Sim	Classe Ente, Personagens, Obstáculos
5.4	- Coesão/Desacoplamento efetiva e intensa inclusive com o apoio de mais de 5 padrões de projeto.	Parcialmente.	Apenas um padrão de projeto completamente implementado até agora.
6	Organizadores e Estáticos		
6.1	- Espaço de Nomes (<i>Namespace</i>) criada pelos autores.	Sim	Namespace Stalingrado, que engloba todos os outros namespaces dentro de si.
6.2	- Classes aninhadas (<i>Nested</i>) criada ou adaptadas pelos autores.	Sim	Classe Lista possui a classe Elemento aninhada
6.3	- Atributos estáticos e métodos estáticos.	Sim	Atributo dt estático, método getDt estático
6.4	- Uso extensivo de constante (<i>const</i>) parâmetro, retorno, método...	Sim	Em vários getters
7	Standard Template Library (STL) e String OO		
7.1	- A classe Pré-definida <i>String</i> ou equivalente. & - <i>Vector</i> e/ou <i>List</i> da <i>STL</i> (p/ objetos ou ponteiros de objetos de classes definidos pelos autores)	Sim	Presente no Gerenciador gráfico, para armazenamento dos nomes das texturas. Bem como presentes também no gerenciador de colisões
7.2	- Pilha, Fila, Bifila, Fila de Prioridade, Conjunto, Multi-Conjunto, Mapa OU Multi-Mapa.	Sim	Mapa utilizado para guardar as texturas dos personagens, obstáculos, chão e fundo.
---	Programação concorrente		
7.3	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos, utilizando Posix, C-Run-Time OU Win32API ou afins.	Não	
7.4	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos com uso de Mutex, Semáforos, OU Troca de mensagens.	Não	...
8	Biblioteca Gráfica / Visual		
8.1 &	- Funcionalidades Elementares. - Funcionalidades Avançadas como: tratamento de colisões, duplo <i>buffer</i> ou outros.	Parcial	Tratamento de colisões
8.2 OU	- Programação orientada e evento efetiva (com gerenciador apropriado de eventos inclusive, via padrão de projeto <i>Observer</i>) em algum ambiente gráfico. - <i>RAD</i> – <i>Rapid Application Development</i> (Objetos gráficos como formulários, botões etc).	Não	
---	Interdisciplinaridades via utilização de Conceitos de Matemática Contínua e/ou Física.		
8.3	- Ensino Médio Efetivamente.	Sim	Cinemática e mecânica básica
8.4	- Ensino Superior Efetivamente.	Sim	Conceitos de geometria analítica na utilização do projétil
9	Engenharia de Software		
9.1	- Compreensão, melhoria e rastreabilidade de cumprimento de requisitos.	parcial	...

9.2	- Diagrama de Classes em <i>UML</i> .	parcial, não completo	...
9.3	- Uso efetivo e intensivo de padrões de projeto <i>GOF</i> , <i>i.e.</i> , mais de 5 padrões.	Parcial	Apenas o Singleton está completamente implementado.
9.4	- Testes à luz da Tabela de Requisitos e do Diagrama de Classes.	Parcial	...
10 Execução de Projeto			
10.1 &	- Controle de versão de modelos e códigos automatizado (via github). - Uso de alguma forma de cópia de segurança (<i>i.e.</i> , <i>backup</i>).	Sim	GitHub, versionamento em branches para backup https://github.com/TinselMoon/stalingrado
10.2	- Reuniões com o professor para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA DO TRABALHO]	não	Especificar quantidade e quando , sendo o mínimo de 2 reuniões a serem marcadas conforme instrução a ser dada (normalmente via planilha <i>on line</i> a ser indicada). <i>Estas duas reuniões só depois das 4 reuniões com os monitores / Peteco (vide item justo abaixo). Ainda, após cada reunião, enviar e-mail para o professor, com cópia para seu colega de dupla, relatando sucintamente a reunião.</i>
10.3	- Reuniões com monitor da disciplina para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA DO TRABALHO]	Sim	Deve haver 4 reuniões com o monitor (de algo com meia-hora cada) e/ou curso com o pessoal do PETECO (de algo com duas horas pelo menos), especificando quando (i.e. data e hora!) . <i>Ainda, após cada reunião ou curso, enviar e-mail para o professor, com cópia para seu colega de dupla e monitor, relatando sucintamente a reunião/curso.</i>
10.4 &	- Escrita do trabalho e feitura da apresentação - Revisão do trabalho escrito de outra equipe e vice-versa.	não	Especificar quem fez o quê. Especificar qual equipe.
Total de conceitos apropriadamente utilizados. (Cada grande tópico vale 10% do total de conceitos. Assim, caso se tenha feito metade de um tópico, então ele valeria 5%.)			70% (setenta por cento). (Naturalmente e obviamente, este tipo de observação aqui entre parênteses deve ser retirada dos relatórios.)